

Universidad Rafael Landívar.
Facultad de Ingeniería.
Ingeniería en Informática y Sistemas
Pensamiento Computacional, Sección 15.
Docente: Ing. Cindy García

LABORATORIO # 15
Control De Calidad De Las Soluciones

Estudiante: Pinzón Castillo, Damaris Azucena
Carné: 1188926

Guatemala, 17 de Mayo de 2026.

1. TIPO DE DEFECTOS

a. Un programa en C# para la recopilación de datos personales muestra el siguiente mensaje en pantalla:

“Ingresar su fecha de nacimiento y presionar Enter.”

Como resultado, el sistema ha sido incapaz de interpretar las respuestas de los usuarios debido a que todas vienen en formatos distintos (ej. DD/MM/YYYY, MM-DDYYYY, YYYY-MM-DD, etc.)

- Defecto de Ambigüedad.

- Defecto de Inconsistencia.
- Defecto Lógico.
- Defecto matemático.

b. Se ha desarrollado un programa que calcula la suma, resta producto y cociente entre dos números ingresados por el usuario. Sin embargo, el programa presenta defectos cuando el usuario ingresa 0 en ambos valores pues no existe la división entre cero.

- Defecto de Escritura.
- Defecto matemático.
- Defecto de Ambigüedad.
- Defecto de Inconsistencia.

c. Se solicita desarrollar un programa que genere 20 números pares aleatorios entre 50 y 100. Deberá mostrar el listado de números en pantalla, resaltando en color cyan los números primos, y calcular las medidas de tendencia central.

- Defecto de Ambigüedad.
- Defecto de Escritura.
- Defecto matemático.
- Defecto de Inconsistencia.

d. Un sistema bancario en línea requiere que la contraseña de sus usuarios contenga entre 8 y 16 caracteres. A pesar de que se implementó una validación recientemente, se han detectado múltiples incidencias de fallo.

```
if (longitud >= 8 || longitud <= 16) {  
    AceptarContraseña(contraseña)  
} else{  
    RechazarContraseña(contraseña)  
}
```

- Defecto de Ambigüedad.
- Defecto de Escritura.
- Defecto lógico.
- Defecto de Inconsistencia.

2. PRUEBAS DE CALIDAD (TESTING) – BOTTOM UP

Elabore un plan de pruebas para validar el correcto funcionamiento de los siguientes métodos. Para cada método deberá realizar tres pruebas, indicando:

- Descripción de la prueba
- Valores de entrada
- Resultado Esperado
- Resultado obtenido, si el método cumple o no la prueba.

El objetivo del ejercicio es evidenciar, si los hay, los defectos en cada método y sugerir una propuesta de mejora.

- a. El método `RetirarEfectivo()` permite retirar dinero de una cuenta bancaria por medio de un cajero automático. Esta recibe dos parámetros; El saldo disponible en la cuenta y el monto que desea retirar.

```
0 referencias
static void RetirarEfectivo(ref double saldo, double montoRetirado) {
    saldo = saldo - montoRetirado;
}
```

CASO	ENTRADA	RESULTADO ESPERADO	RESULTADO OBTENIDO
Retiro mayor a saldo	Saldo: 200 Retiro: 300	Mensaje de error, no se realizó el retiro	Q -100
Retiro negativo	Saldo: 700 Retiro: -400	Mensaje de error. El monto debe ser mayor a cero.	Q1100
Saldo negativo	Saldo: -100 Retiro: 200	Mensaje de error. Error de cuenta.	Q -300

PROPUESTA

Agregar una validación para impedir que el retiro sea mayor al saldo en la cuenta. Implementar otra validación para evitar la resta de un negativo, y que se convierta en una suma. Por último, agregar otra validación para evitar que el saldo en la cuenta sea negativo, para bloquear las transacciones.

- b. La función `CalcularDescuento()` permite aplicar un descuento o rebaja al precio original de un producto. Para esto utiliza dos entradas; el precio original del producto y el porcentaje de descuento, entre 0 y 100, que debemos aplicar.

```
0 referencias
static double CalcularDescuento(double precio, double porcentaje)
{
    return precio - (precio * porcentaje / 100);
}
```

CASO	ENTRADA	RESULTADO ESPERADO	RESULTADO OBTENIDO
Descuento negativo	Precio: 500 Descuento: -50	Mensaje de error	Q 750
Porcentaje mayor a 100	Precio: 500 Descuento: 120	Mensaje de error	Q -100
Precio negativo	Precio: -100 Descuento: 10	Mensaje de error	Q-90

PROPUESTA

Añadir una validación donde si el descuento es negativo, el sistema ignore la rebaja y lo trate como 0%, sin descuento, para evitar que el precio final aumente.

Añadir otra validación donde si el descuento supera los 100% el sistema limite el resultado a cero, obteniendo un descuento total, para así evitar que el comercio termine con un saldo negativo.

Además, añadir otra validación donde si el precio es negativo, se identifique como un error de entrada.

- c. El método Depositar () permite aumentar el saldo de una cuenta bancaria. Este recibe como entrada el saldo disponible del cliente y el monto que desea depositar.

```
0 referencias
static void Depositar(ref double saldo, double monto)
{
    saldo = saldo + monto;
}
```

CASO	ENTRADA	RESULTADO ESPERADO	RESULTADO OBTENIDO
Depósitos negativos	Saldo: 400 Monto: -100	Mensaje de error.	300
Depósito válido	Saldo: 600 Monto: 200	El saldo final es 800	800
Saldo negativo	Saldo: -700 Monto: 100	Mensaje de error	-600

PROPUESTA

Agregar una validación al programa para evitar errores al momento de realizar un depósito. El sistema debe verificar que el saldo no sea negativo y que el monto a depositar no sea negativo. Si los datos son incorrectos, la transacción se cancela y se muestra un mensaje de error. En cambio, si los datos son correctos el depósito se realiza y se muestra el resultado.

3. SEGUIMIENTO PASO A PASO (DEBUGGING)

Crear un proyecto de consola en C# e ingresar el código proporcionado para simular el pago de un crédito personal.

```
static void Main()
{
    double capital = 1000;
    double tasa = 0.05;
    double intereses = 0;
    double abonos = 0;

    for (int mes = 1; mes <= 12 && capital > 0; mes++)
    {
        // Cálculo de intereses del mes
        intereses = capital * tasa;

        // Abono realizado cada mes
        abonos = 100 + (mes * 10);

        // Actualización del capital
        capital = capital + intereses - abonos;
    }
}
```

Deberá utilizar herramientas de depuración (debugging) mediante el uso de breakpoints para observar cómo cambian las variables durante la ejecución del programa.
Instrucciones:

- a. Ejecute el programa en modo depuración (Debug).
- b. Coloque un breakpoint dentro del ciclo for, específicamente en la línea donde se actualiza el capital.
- c. Utilice las herramientas de depuración:
 - Step Over (F10)
 - Ventana de variables locales
- d. Observe cómo cambian las siguientes variables durante cada iteración:
 - capital
 - intereses
 - abonos
 - tasa
 - mes
- e. Tome capturas de pantalla donde se visualicen claramente los valores de las variables al finalizar:

MES 2

Program.cs

```
12
13
14 //3. SEGUIMIENTO PASO A PASO (DEBUGGING)
15 Console.WriteLine();
16 Console.WriteLine(" - EJERCICIO # 03 - ");
17 Console.WriteLine(" SEGUIMIENTO PASO A PASO (DEBUGGING) ");
18
19 double capital = 1000;
20 double tasa = 0.05;
21 double intereses = 0;
22 double abonos = 0;
23 for (int mes = 1; mes <= 12 && capital > 0; mes++)
24 {
25     //calculo intereses del mes
26     intereses = capital * tasa;
27
28     //abono realizado cada mes
29     abonos = 100 + (mes * 10);
30
31     //actualizar el capital
32     capital = capital + intereses - abonos;
33 }
34
35
36
37
```

Variables locales

Nombre	Valor	Tipo
capital	867	double
tasa	0.05	double
intereses	47	double
abonos	120	double
mes	2	int

Herramientas de diagnóstico

Sesión de diagnóstico: 1 segundos (1.48 s selección...)

1.486s

Eventos

Memoria de proceso (MB)

CPU (% de todos los procesadores)

Resumen | Eventos | Uso de memoria | Uso de CPU

Eventos

Mostrar eventos (9 de 9)

Uso de memoria

Tomar instantánea

Uso de CPU

Pila de llamadas

Nombre

L15 - PINZON CASTILLO DAMARIS AZUCENA 1188926.exel.L15_PINZON_CASTILLO DAMARIS AZUCENA 1188926.Program.Mai... C#

Variables locales

Buscar (Ctrl+E)

Profundidad de búsqueda: 3

Nombre	Valor	Tipo
capital	867	double
tasa	0.05	double
intereses	47	double
abonos	120	double
mes	2	int

Variables locales | Inspección 1

MES 4

The screenshot shows the Visual Studio Code interface with a C# program being debugged. The program is a simulation of a loan calculation over 12 months. The current state of the program is at line 32, where the variable `mes` is 4. The diagnostic tools on the right show a session of 1.496 seconds, with memory usage at 8 MB and CPU usage at 0%.

```
12 {
13     //3. SEGUIMIENTO PASO A PASO (DEBUGGING)
14     Console.WriteLine();
15     Console.WriteLine(" - EJERCICIO # 03 - ");
16     Console.WriteLine(" SEGUIMIENTO PASO A PASO (DEBUGGING) ");
17
18     double capital = 1000;
19     double tasa = 0.05;
20     double intereses = 0;
21     double abonos = 0;
22     for (int mes = 1; mes <= 12 && capital > 0; mes++)    mes: 4
23     {
24         //calculo intereses del mes
25         intereses = capital * tasa;
26
27         //abono realizado cada mes
28         abonos = 100 + (mes * 10);
29
30         //actualizar el capital
31         capital = capital + intereses - abonos;
32     }
33 }
34
35
36
37
```

Variables locales

Nombre	Valor	Tipo
capital	679.36750000000006	double
tasa	0.05	double
intereses	39.017500000000005	double
abonos	140	double
mes	4	int

Pila de llamadas

Nombre	Leng
L15 - PINZON CASTILLO DAMARIS AZUCENA 1188926.exe	C#

This is a close-up of the Variables Locales window. It shows the current state of the program's variables. The search bar is empty, and the depth of search is set to 3. The variables are listed in a table with their names, values, and types.

Nombre	Valor	Tipo
capital	679.36750000000006	double
tasa	0.05	double
intereses	39.017500000000005	double
abonos	140	double
mes	4	int

Variables locales Inspección 1

Listo

MES 6

The screenshot shows the Visual Studio Code interface with a C# program open. The program is titled "L15 - PINZON CASTILLO DAMARIS AZUCENA 1188926". The code is as follows:

```
12 {  
13     //3. SEGUIMIENTO PASO A PASO (DEBUGGING)  
14     Console.WriteLine();  
15     Console.WriteLine(" - EJERCICIO # 03 - ");  
16     Console.WriteLine(" SEGUIMIENTO PASO A PASO (DEBUGGING) ");  
17  
18     double capital = 1000;  
19     double tasa = 0.05;  
20     double intereses = 0;  
21     double abonos = 0;  
22     for (int mes = 1; mes <= 12 && capital > 0; mes++)    mes: 6  
23     {  
24         //cálculo intereses del mes  
25         intereses = capital * tasa;  
26  
27         //abono realizado cada mes  
28         abonos = 100 + (mes * 10);  
29  
30         //actualizar el capital  
31         capital = capital + intereses - abonos;  
32     }  
33 }  
34  
35  
36  
37
```

The status bar at the bottom indicates "100 %", "No se encontraron problemas.", and "Línea: 32, Carácter: 13 SPC CRLF UTF-8 with BOM".

On the right side, the "Herramientas de diagnóstico" (Diagnostic Tools) window is open, showing a "Sesión de diagnóstico: 1 segundos (1.503 s selecci...)" and various performance metrics like "Eventos", "Memoria de proceso (MB)", and "CPU (% de todos los procesadores)".

At the bottom, the "Variables locales" (Local Variables) window is open, showing the following variables:

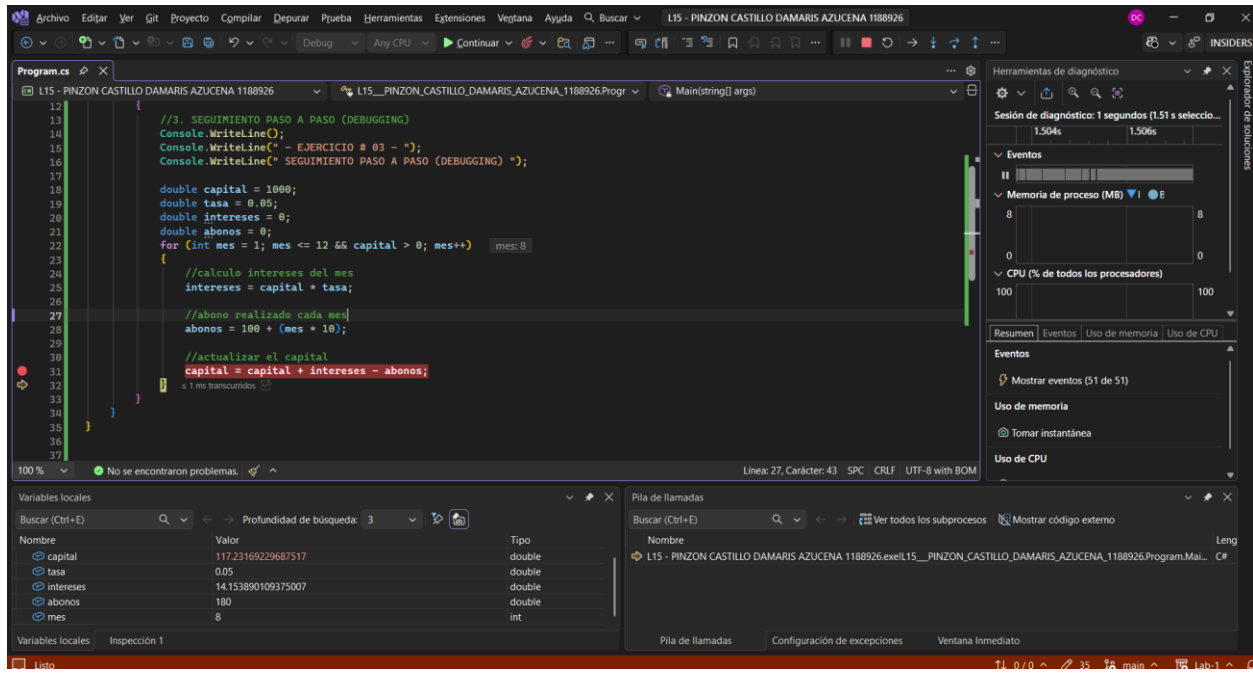
Nombre	Valor	Tipo
capital	431.50266875000011	double
tasa	0.05	double
intereses	28.166793750000007	double
abonos	160	double
mes	6	int

This is a close-up screenshot of the "Variables locales" window. It shows the same variables as the previous screenshot:

Nombre	Valor	Tipo
capital	431.50266875000011	double
tasa	0.05	double
intereses	28.166793750000007	double
abonos	160	double
mes	6	int

The window title is "Variables locales" and the status bar at the bottom shows "Listo".

MES 8

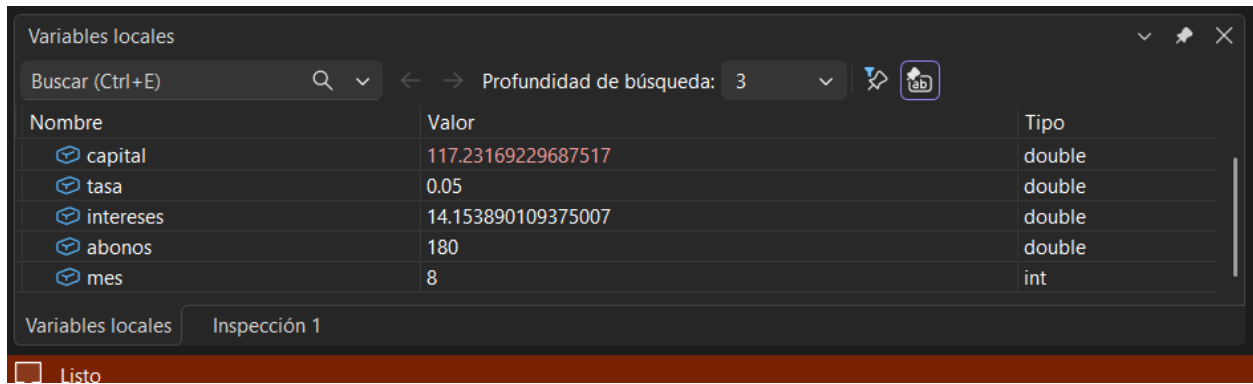


The screenshot shows a Visual Studio Code editor with a C# program for a loan simulation. The program is titled "L15 - PINZON CASTILLO DAMARIS AZUCENA 1188926". The code is as follows:

```
13 {  
14     //3. SEGUIMIENTO PASO A PASO (DEBUGGING)  
15     Console.WriteLine();  
16     Console.WriteLine(" - EJERCICIO # 03 - ");  
17     Console.WriteLine(" SEGUIMIENTO PASO A PASO (DEBUGGING) ");  
18  
19     double capital = 1000;  
20     double tasa = 0.05;  
21     double intereses = 0;  
22     double abonos = 0;  
23     for (int mes = 1; mes <= 12 && capital > 0; mes++)  
24     {  
25         //calculo intereses del mes  
26         intereses = capital * tasa;  
27         //abono realizado cada mes  
28         abonos = 100 + (mes * 10);  
29  
30         //actualizar el capital  
31         capital = capital + intereses - abonos;  
32     }  
33 }  
34  
35  
36  
37
```

The 'Variables locales' window shows the current state of variables:

Nombre	Valor	Tipo
capital	117.23169229687517	double
tasa	0.05	double
intereses	14.153890109375007	double
abonos	180	double
mes	8	int



The close-up screenshot shows the 'Variables locales' window with the following data:

Nombre	Valor	Tipo
capital	117.23169229687517	double
tasa	0.05	double
intereses	14.153890109375007	double
abonos	180	double
mes	8	int